

Track change AGQ implementation

Lucie Fayette

May 18, 2026

- Now, observed FIM for each MC sample are kept; would be used to compute confidence interval using bootstrap
- The solver to find the mode of the random effects has been changed
- Minor edits

Keeping observed FIM for each MC sample

`integrate_mc`

in the script `integration.R`, the function `integrate_mc` has been changed to return results for all the sample and not only the average

```
integrate_mc <- function(fun, dimensions, settings=filter_settings(defaults.agq(), "mc"), ...){
  additional.args <- c(list(), list(...))
  n.samples <- settings$n_samples
  if(is.character(dimensions)) {
    ndim <- length(dimensions)
  }else{
    ndim <- dimensions
  }
  if(ndim>0){
    param.samples <- matrix(rnorm(n.samples*ndim, 0, 1), ncol=ndim)
  }else{
    param.samples <- matrix(nrow=n.samples)
  }
  if(is.character(dimensions)) {
    colnames(param.samples) <- dimensions
  }
  call.args <- c(additional.args, list(param.samples))
  mc.results <- do.call(fun, call.args)
  if(!is.matrix(mc.results)) return(mean(mc.results))
  return(rowMeans(mc.results))
}
```

becomes

```
integrate_mc <- function(fun, dimensions, settings=filter_settings(defaults.agq(), "mc"), ...){
  additional.args <- c(list(), list(...))
  n.samples <- settings$n_samples
  if(is.character(dimensions)) {
    ndim <- length(dimensions)
```

```

}else{
  ndim <- dimensions
}
if(ndim>0){
  param.samples <- matrix(rnorm(n.samples*ndim, 0, 1), ncol=ndim)
}else{
  param.samples <- matrix(nrow=n.samples)
}
if(is.character(dimensions)) {
  colnames(param.samples) <- dimensions
}
call.args <- c(additional.args, list(param.samples))
mc.results <- do.call(fun, call.args)
if(!is.matrix(mc.results)) return(mean(mc.results))
return(list(res = rowMeans(mc.results),
  all_y = mc.results))
}

```

fim_std_sampling

l23 in the Model object definition, the function fim_std_sampling

```

fim_std_sampling = function(design){
  matrix(integrate_mc(function(eta.vec){
    apply(eta.vec,1,function(eta){
      sim <-simulation_eta_function(design, eta)
      observed_fim(sim, design, eta)
    })
  }, re_count, settings=settings_y_integration), nrow=theta_count)
}

```

becomes

```

fim_std_sampling = function(design){

  res_MC = integrate_mc_LF(function(eta.vec){
    apply(eta.vec,1,function(eta){
      sim <-simulation_eta_function(design, eta)
      observed_fim(sim, design, eta)
    })
  }, re_count, settings=settings_y_integration)
  FIM = matrix(res_MC$res, nrow=theta_count)
  all_y = res_MC$all_y

  return(list(FIM = FIM,
    all_y = all_y))
}

```

fim

l347 in the Model object definition, the function fim

```
fim = function(design){
  fim <- calc_fim(design)
  rownames(fim) <- theta_names
  colnames(fim) <- theta_names
  return(fim)
}
```

changes to

```
fim = function(design){
  fim <- calc_fim(design)
  rownames(fim$FIM) <- theta_names
  colnames(fim$FIM) <- theta_names
  return(list(FIM = fim$FIM,
             all_y = fim$all_y))
}
```

Optimisation solver

l255 in the Model object definition, the function gradient_joint_likelihood

```
dLLdP <- do.call(gradient_dll_dpar, c(list(y, design), parameter_function(mu,b)))
```

to

```
dLLdP <- do.call(gradient_dll_dpar, c(list(y=y, design=design), parameter_function(mu,b)))
```

Minor edits

l312 in the Model object definition, the function observed_fim

```
res <- optim(eta_sim, f, method = "BFGS", hessian = T)
```

to

```
res = try(optim(eta_sim, f, method = "Nelder-Mead", hessian = T))
```

l341 in the Model object definition, the function observed_fim

```
if(is.na(ml) || ml==0 || is.nan(gradient))
```

to

```
if(is.na(ml) || ml==0 || is.nan(sum(gradient)))
```
